

TraceMeet

Technical description

Every meeting. A record you can trace.

github.com/adityaomsah/TraceMeet

Objective and scope

TraceMeet converts an English meeting recording into a reviewable transcript and structured meeting documentation. The application uses three model roles, with Python validation and file-based orchestration between them. It is designed for local CPU transcription and cloud language-model inference through a Streamlit UI.

Model roles

Stage	Configured implementation	Contract
Transcription	faster-whisper small.en, CPU/int8	Stable segment IDs, start/end seconds and raw text
Refinement	Gemini gemini-3.5-flash-lite	Revised text for the same ordered segment IDs
Documentation	Groq-hosted openai/gpt-oss-120b	Summary, topics, decisions, tasks and open questions

Model identifiers describe the development configuration. Access is checked with small structured preflight requests; visibility in a provider's model list alone does not demonstrate successful inference.

Data flow and safeguards

- 1. Input validation.** PyAV checks that the media opens and has an audio stream; empty, missing and too-short inputs are rejected. Structural validation does not establish that speech exists. Whisper/VAD handles transcription.
- 2. Raw transcript.** Local speech recognition produces timestamped segments. The raw transcript is retained for comparison. Speaker diarization is not implemented; segment IDs are not speaker identities.
- 3. Refinement proposals.** The first LLM receives segment text and optional terminology context. Chunked calls use neighboring context and checkpoints. Python validates target identity/order rather than accepting dropped, duplicated or reordered segment outputs.
- 4. Correction guards.** Python computes word/token changes and character offsets. Whole-segment heuristics inspect sensitive markers: numbers, negation, commitments, dates, units, currency and protected/capitalized names. A flagged segment retains its raw wording. The UI exposes the proposal and reason; it does not currently implement human approval of edits.

5. Meeting documentation. The second LLM receives the guarded transcript. Short requests use the meeting-record schema directly. Larger requests use the long workflow described below. Owners and deadlines require dedicated evidence fields; missing values remain null.

6. Citation checks. Each quote must be an exact contiguous substring of its referenced segment with word-boundary checks. Invalid primary evidence can drop an item; invalid owner/deadline evidence can clear that field. The report explicitly distinguishes citation presence from semantic support.

7. Presentation and exports. The UI and Markdown/JSON/task-CSV/ZIP exports use the same citation-checked record. Playback resolves segment timestamps to the retained recording. The overview remains uncited under the current schema.

Long-meeting workflow

The router estimates input tokens, schema/message overhead and an output reserve. The configured 7,400-token request budget is an application constraint chosen for the tested account, not a universal Groq context-window or quota limit.

Map: windows produce provisional observations citing global segment IDs. Python records supplied input coverage and retrieves source wording. It does not ask the model to prove completeness by echoing every input ID.

Group: observations from across the meeting are assigned to topic groups. Structural checks ensure every observation receives one assignment. This does not prove that semantically related material was grouped correctly.

Reconcile: topic groups are examined against source segments to produce candidate documentation. Later rejection, reassignment and changed deadlines must be considered; intermediate observations are not final decisions.

Finalize: candidate review uses explicit dispositions and budgeted batches. Overview generation is bounded by both input and output allowances, accounting for source topics and declared omissions. Completed substeps are checkpointed. Structural coverage checks do not guarantee semantic completeness or cross-group consistency.

Reliability and provenance

Run folders store original media, metadata, transcripts, candidate/checked records, correction and citation reports, and long-stage checkpoints. Fingerprints include relevant inputs, configuration and module/prompt files. Stages are reused only when the recorded artifacts match their expected hashes; affected downstream stages are invalidated after changes.

The pipeline uses atomic temporary-file replacement and a per-run writer lock. Provider clients are created only when needed. Long Groq processing shares a pacing clock across stages; pacing does not coordinate separate processes using the same account. Error recovery preserves valid work. A developer-observed Windows access-denied error during state replacement recovered after restart and resume; its cause and permanent resolution are not established.

Legacy artifacts can be adopted only under their metadata checks. This does not retroactively establish the historical source-code version. Display titles are stored separately and do not affect inference or folder identity.

Why the documentation provider changed

The initial implementation used Gemini for both language-model roles. Development logs showed schema-related HTTP 400 errors, intermittent 503 responses, quota failures and one cancelled request. Successful small probes did not consistently predict success on the longer recording.

Schema transport was adjusted to use JSON Schema, and bounded retries/checkpoints were introduced. Groq's GPT-OSS model then passed a synthetic structured extraction probe and a short real-recording test. Documentation moved to Groq while the working Gemini refinement stage was retained. This was an operational engineering choice, not a controlled proof that one model is universally more accurate.

Groq introduced its own request-budget, truncation and schema-validation failures. The long workflow, strict local validation, source-ID citations and explicit candidate dispositions address those observed failure modes. Automatic cross-provider fallback and local Ollama inference are not part of the integrated profile.

Verification and limitations

The developer-reported test run at revision 7014be9 passed all 246 tests in 6.33 seconds. This run was not repeated during document preparation. Live short recordings exercised the complete application, including playback and exports. A 463-segment long recording exercised staged extraction through finalization. These are functional observations, not accuracy benchmarks. Annotated evaluation, held-out testing and the raw/refined ablation remain pending.

Main limitations are heuristic edit protection, uncertain speaker attribution, uncited summaries, incomplete semantic verification, potential loss of context in multi-stage processing, cloud nondeterminism and quota/latency dependence. Audit trails support investigation; they do not make generated claims automatically true.